

Московский авиационный институт (национальный исследовательский университет)

Институт «Компьютерные науки и прикладная математика»

Кафедра «Вычислительная математика и программирование»

Лабораторная работа №3

по курсу «Дискретный анализ»

на тему «Профилирование»

Студент: Чигрин Никита Александрович

Группа: М8О-208БВ-24

Преподаватель: Макаров Н.К.

Проверил: Сахарин Н.А.

Дата: 11 апреля 2026 г.

Оценка: _____

Подпись: _____

Москва, 2026

Постановка задачи

Провести профилирование программы-словаря с помощью инструментов gprof, callgrind, perf, valgrind, dmalloc.

Цели анализа

- Выявить утечки памяти и ошибки работы с динамической памятью;
- Найти узкие места производительности (системные вызовы, копирование строк, балансировка и т.д.);
- Обнаружить неоптимальные алгоритмы в коде.

Используемые инструменты

Инструмент	Назначение
gprof	Профилирование по времени и числу вызовов функций
callgrind	Детальный подсчёт инструкций, моделирование кэша, граф вызовов
perf	Анализ работы ядра, системных вызовов, прерываний
valgrind	Проверка памяти (чтение/запись, утечки, неинициализированные значения)
dmalloc	Детальный учёт выделений, поиск утечек

Тестирование

Профилирование и анализ программы на возможные проблемы с памятью проводились на тестовых данных, приведённых в таблице 1.

Таблица 1: Результаты тестирования

Команда	Ожидаемый вывод	Реальный вывод
+ a 1	OK	OK
a	OK: 1	OK: 1
+ A 20	Exist	Exist
- a	OK	OK
A	NoSuchWord	NoSuchWord

Профилирование с gprof

Запуск

./avltree # (название исполняемого файла предположительно)

```
gprof avltree gmon.out > gprof_report.txt
```

Результаты

Время выполнения: Отчёт показывает no time accumulated (время не накоплено). Это означает, что либо программа выполнялась очень короткое время (меньше минимального шага сэмплирования в 0.01 секунды), либо тестовые данные были слишком малы, чтобы профилировщик зафиксировал значимые затраты.

Ключевые вызовы: Несмотря на нулевое время, видна структура вызовов:

- Конструкторы/деструкторы string: 39 вызовов (string), 32 вызова (string(const char*)).
- Операторы сравнения строк: 20 вызовов operator==, 17 вызовов operator<.
- Операции ввода: 12 вызовов operator» (чтение строк).
- AVL-дерево: 2 вставки (add), 2 поиска (find), 1 удаление (del).

Вспомогательные функции: Много вызовов std::move, std::forward, std::get и конструкторов std::pair — следствие активного использования семантики перемещения и шаблонов.

Вывод по gprof

Узким местом являются частые копирования (через конструкторы) и сравнения строк (operator<, operator==). AVL-дерево используется мало (3 вставки / 3 поиска). Для получения реальных временных затрат необходимо увеличить объем тестовых данных (больше вставок, поисков и удалений), чтобы превысить порог сэмплирования в 0.01 секунды.

Профилирование с perf

Сбор событий: perf record ./main

Просмотр отчёта: perf report

Результаты:

- 53.23% времени процессора — динамическая линковка (_dl_lookup_symbol_x, do_lookup_x, memset)
- 31.34% — ядро Linux (неразрешённые символы)
- 15.21% — libstdc++ (неразрешённый символ)

Из них:

- 26.81% — _dl_lookup_symbol_x (поиск символов)
- 16.25% — ядро (смещение 0xffffffffb307b180)

- 11.21% — `do_lookup_x` (вспомогательный поиск)
- 7.79% — `memset`

Вывод: программа чрезмерно использует динамическую линковку (более половины времени)

Профилирование с Callgrind

Запуск

```
1 valgrind --tool=callgrind ./main
```

Генерация отчета

```
1 callgrind_annotate callgrind.out.11781 > callgrind_report.txt
```

Результаты

- **Всего инструкций:** 2 021 720.
- **Наиболее затратные функции (по кол-ву инструкций):**
 - `main` (116 172 инструкции).
 - `string::operator<` (26 337 инструкций) внутри `AvlTree<string...>::find`.
 - Инициализация глобальных объектов C++ (например, `std::ios_base::Init::Init`).
 - `string::operator==` (6 851 инструкция) и `string::operator<` (26 337 инструкций) внутри `AvlTree<string...>::find`.

Вывод

- Профиль указывает на то, что основное время выполнения программы (как и в прошлом анализе с `gprof`) тратится на операции сравнения строк (`operator<`, `operator==`) в методах АВЛ-дерева (`find`, `add`, `del`).
- Значительная часть инструкций уходит на инициализацию стандартной библиотеки C++ (локали, потоки ввода-вывода)

Анализ памяти с Valgrind

Запуск

```
valgrind ./main
```

Результаты

- Обнаружено 2 ошибки от 1 контекста.
- **Ошибка:** Conditional jump or move depends on uninitialised value в конструкторе `string::string(char const*)` и операторе `operator»`.
- Неинициализированное значение создано при стековой аллокации в `operator»`.
- **Утечки:** 122 880 байт в 6 блоках — все still reachable (буферы ввода-вывода, статические буферы), definitely lost = 0 байт.
- Функциональность программы работает корректно (вывод "OK", "Exist", "NoSuchWord").

Заключение

Ошибки не критические, связаны с чтением неинициализированной памяти при вводе строк. Утечек памяти нет.

Анализ памяти с `dmalloc`

Запуск

```
dmalloc -l logfile -i 100 low
./main
dmalloc -l logfile -i 100 high
```

Общая характеристика

- Программа не набирает значительного времени выполнения ни в одной из функций (“no time accumulated”, все значения 0.00 секунд).
- Это характерно для программ с очень малым временем работы (менее 0.01 секунды), где точность профилировщика недостаточна для захвата измерений.

Структура вызовов

Работа с AVL-деревом:

- Выполнено 2 операции добавления (`AvlTree::add`).
- Выполнено 2 операции поиска (`AvlTree::find`).
- Выполнена 1 операция удаления (`AvlTree::del`).
- Создано 2 узла дерева (`AvlTree::Node::Node`).
- Уничтожено 2 узла (`AvlTree::Node::Node`).

Работа со строками:

- Создано 32 строки через конструктор от `const char*`.
- Выполнено 39 деструкторов строк.
- 20 операций сравнения строк (`operator==`).
- 17 операций сравнения “меньше” (`operator<`).
- 12 операций перемещающего присваивания.

Балансировка дерева:

- Выполнена 1 операция балансировки после добавления (`balance_add`).
- 17 вызовов компаратора `std::less<string>::operator()`.

Вывод о программе

Программа написана качественно, но может быть дополнительно оптимизирована. Профилирование помогло выявить не ошибки, а резервы производительности